

MERN STACK INTERNSHIP – DAY 1 ASSIGNMENT REPORT

Company

Insta Dot Analytics

Submitted By

Name: Chayan Soni

Roll No: PU02325EUGBTCS086

Course: B.Tech (CSIT), First Year

College: Symbiosis University of Applied Sciences

Submission Date: 02-06-2026

SECTION 1: ASSIGNMENT OVERVIEW

The Day 2 assignment focused on introducing modern backend development concepts using Node.js and Express.js. The objective was to build a functional server capable of handling asynchronous operations, reading local files, processing data, and exposing the results through a REST API endpoint. In addition to backend development, the assignment also explored client-server communication using the JavaScript Fetch API. A frontend interface was developed to consume the API endpoint and display the returned data through the browser console.

This assignment helped establish a strong understanding of asynchronous programming, Express routing, file system operations, and frontend-backend integration.

Technologies & Tools Used

Category	Technologies
Backend	Node.js, Express.js
Frontend	HTML5, JavaScript
Core Modules	fs, path
Data Format	JSON
API Communication	Fetch API
Development Tools	VS Code, Git, GitHub

GitHub Repository: <https://github.com/chayan-soni/MERN-Internship>

SECTION 2: PROJECT STRUCTURE

The Day 2 project was organized into separate files to maintain a clean and scalable codebase.

```
Day2/
|
├── public/
|   └── index.html
|
├── data.json
├── server.js
├── package.json
├── package-lock.json
├── .gitignore
└── README.md
```

Structure Description

File	Purpose
server.js	Express server and API logic
data.json	Local data source
index.html	Frontend testing interface
package.json	Project metadata and dependencies
.gitignore	Git exclusion rules
README.md	Project documentation

SECTION 3: TASK 2.1 – BACKEND ENGINEERING

3.1 Objective

The objective of Task 2.1 was to create a Node.js Express server capable of reading a local JSON file asynchronously, filtering the records, and returning a structured JSON response through an API endpoint.

3.2 Implementation Details

The backend application was developed using Express.js and configured to run on Port 3000.

A dedicated GET route (/users) was created to handle incoming client requests.

When a request is received, the server performs the following operations:

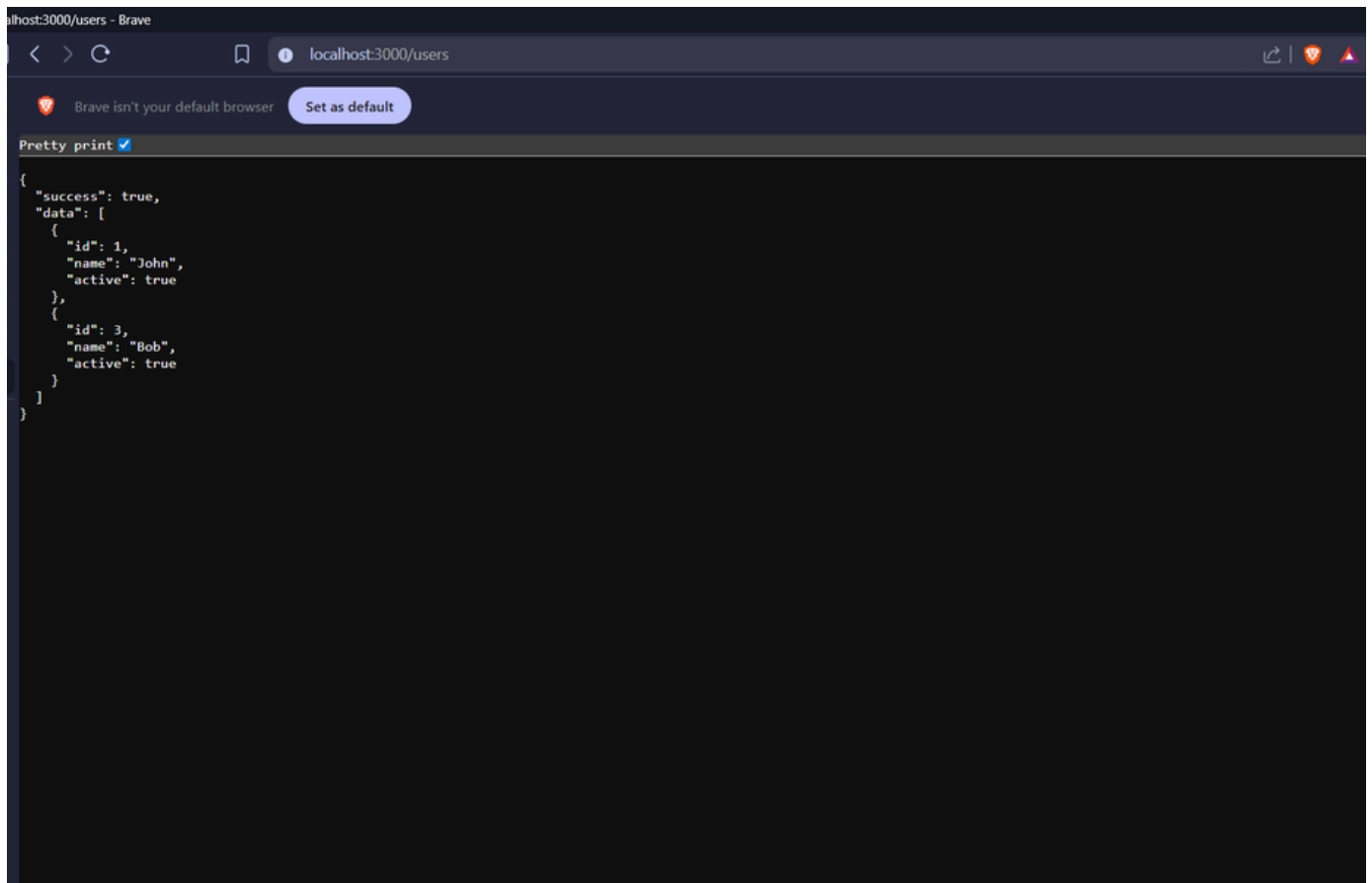
1. Reads the local JSON file asynchronously using the File System module (fs.promises).
2. Resolves the file location using the Path module.
3. Parses the JSON data into JavaScript objects.
4. Filters active users from the dataset.
5. Sends the filtered results back to the client as JSON.

To ensure application stability, all file operations were wrapped inside a try-catch block to handle potential runtime errors gracefully.

3.3 Features Implemented

Feature	Description
Express Server	Local server running on Port 3000
GET Route	/users endpoint
Async/Await	Non-blocking file operations
File System Module	Reading local JSON files
Path Module	Safe file path handling
Data Filtering	Processing user records
JSON Responses	Structured API output
Error Handling	Try-Catch implementation

3.4 Output Screenshot



3.5 GitHub Source Code References

Repository: MERN-Internship

Backend Logic: Day2/server.js

Data File: Day2/data.json

SECTION 4: TASK 2.2 – API CLIENT TESTING

4.1 Objective

The objective of Task 2.2 was to create a frontend client capable of interacting with the backend API and retrieving data asynchronously using the Fetch API.

4.2 Design & Implementation

A simple HTML interface was developed inside the public folder.

An asynchronous JavaScript function was implemented to:

- Send a GET request to the backend endpoint.

- Receive the JSON response.
- Convert the response into a JavaScript object.
- Display the results inside the browser console.

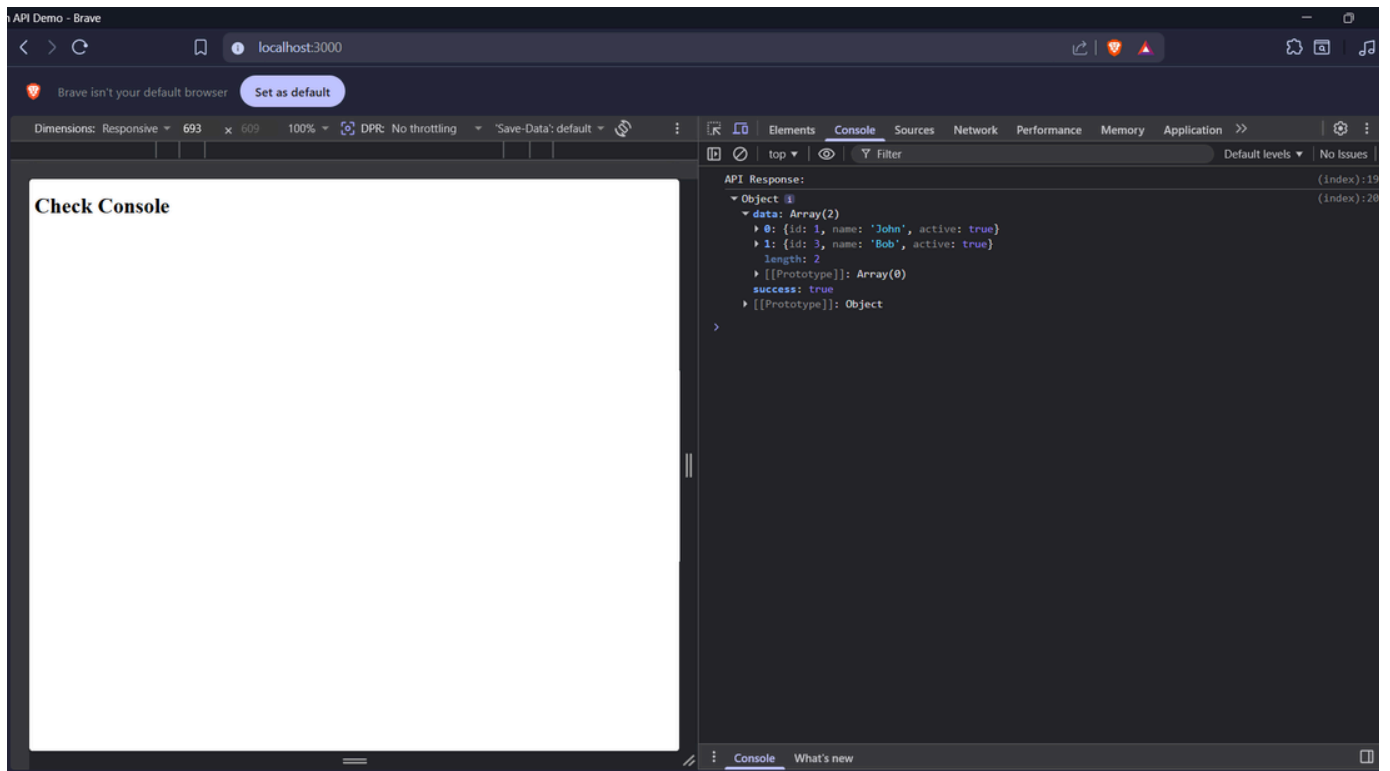
The implementation uses Async/Await syntax for improved readability and better asynchronous control.

Error handling was also implemented using Try-Catch blocks to manage network failures and unexpected issues.

4.3 Features Implemented

Feature	Description
Fetch API	Retrieve backend data
Async Functions	Non-blocking network requests
JSON Parsing	Convert API responses
Console Logging	Display results for testing
Frontend Integration	Communication with backend
Error Handling	Network failure management

4.4 Screenshot Section



4.5 GitHub Source Code References

Frontend Interface: [Day2/public/index.html](#)

SECTION 5: LEARNING OUTCOMES

The Day 2 assignment provided practical exposure to backend development concepts and API communication workflows.

Competency Domain	Skills Acquired
Express.js Fundamentals	Creating and configuring web servers
REST API Development	Designing GET endpoints
Async Programming	Working with Async/Await and Promises
File Handling	Reading and processing local files
Error Management	Implementing Try-Catch blocks
Client-Server Communication	Using Fetch API
JSON Processing	Parsing and filtering data
Git & GitHub	Managing and pushing project code

SECTION 6: CONCLUSION

The Day 2 assignment marked an important step toward understanding modern backend development practices.

By building an Express.js server and implementing asynchronous file operations, valuable experience was gained in handling real-world backend workflows. The assignment demonstrated how data can be retrieved, processed, and delivered through RESTful API endpoints.

Furthermore, implementing the Fetch API on the frontend provided a clear understanding of client-server communication and asynchronous network requests.

Overall, this assignment successfully strengthened core concepts related to Node.js, Express.js, asynchronous programming, file handling, and API integration, creating a strong foundation for future MERN Stack development tasks.

End of Report

Day 2 – MERN Stack Development Internship

Prepared By: Chayan Soni
INSTA Dot Analytics

